THE YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project- High Level Design
on
Cloud-Forensics-Automation-for-Retail-Breaches

Student Name
Joel Abin
23BCCE108

Table of Contents

1. Introduction ----------3
1.1 Scope of the Document----------3
1.2 Intended Audience----------3
1.3 System Overview----------3
2. System Architecture / Design----------4
2.1. Application Design ----------4
2.2. Process Flow ----------4
2.3. Information Flow ----------4
2.4. Components Design ----------5
3. Module Design (High-Level ONLY)----------5
3.1 File Integrity Scanning Module----------5
3.2 Baseline Management Module----------5
3.3 Comparison & Detection Module----------5
3.4 Alert & Risk Detection Module----------5
3.5 Reporting Module----------6
3.6 Dashboard & Visualization Module----------6
4. Data Design----------6
4.1 Data Model----------6
4.2 Data Storage----------6
4.3 Data Retention----------7
5. Interfaces----------7
5.1 User Interface (Dashboard)----------7
5.2 File System Interface----------8
6. Non-Functional Requirements----------8
6.1 Security----------8
6.2 Performance----------8
6.3 Limitations----------8
6.4 Future Enhancements----------9
7. Limitations & Future Enhancements----------9
7.1 Limitations----------9
7.2 Future Enhancements----------9
8. References----------9

1. Introduction

The Cloud-Forensics Automation for Retail Breaches is designed to detect unauthorized modifications in retail data files. In modern retail environments, data integrity is critical as files may be altered intentionally or accidentally, leading to security risks and data inconsistencies.
This system provides a structured approach to monitor file changes, detect tampering, and generate forensic reports. It helps investigators identify suspicious activities by comparing current file states with a previously stored baseline.

1.1 Scope of the Document

This document presents the high-level design of the system. It describes the overall architecture, major components, data flow, and system interactions.
The document covers:

- System architecture and design
- Module overview
- Data handling and storage
- User interface and system interaction
- Non-functional requirements such as security and performance

This document does not include detailed implementation or code-level logic, which are covered in the Low-Level Design (LLD).

1.2 Intended Audience

This document is intended for:

- Project evaluators
- Developers and system designers
- Students and researchers in cybersecurity
- Anyone interested in digital forensics systems

1.3 System Overview

The Retail Digital Forensics System is a web-based application designed to detect unauthorized changes in retail data files.
Retail systems store important business and customer data. During a cyberattack or insider misuse, attackers may modify or delete files without leaving obvious traces. This system helps identify such changes using file integrity monitoring.
The system works by:

- Scanning files in a retail directory
- Extracting file metadata such as name, size, and timestamps
- Creating a baseline of original file states
- Comparing current file data with the baseline
- Detecting changes such as modification, addition, or deletion
- Generating alerts for suspicious activities
- Producing forensic reports in CSV format

The system also includes a simulation module that allows users to perform retail operations (like adding customers) to test tampering detection.
Overall, the system provides a simple and effective approach for detecting file tampering and supporting forensic investigation.

2. System Architecture / Design

2.1 Application Design

The system is designed as a single-tier web application using Streamlit, where both the user interface and backend logic are integrated into a single Python-based system.
The application consists of three main layers:

- User Interface Layer (Frontend)
- Built using Streamlit, this layer provides an interactive dashboard for users to perform operations such as file scanning, baseline creation, simulation, and report generation.
- Application Logic Layer (Backend)
- This layer handles core functionalities including file scanning, metadata extraction, hash generation, comparison with baseline, tampering detection, and alert generation.
- Data Storage Layer
- Stores system data such as:
  - Retail data (CSV/JSON)
  - Baseline hash values
  - Logs and reports

This layered approach ensures separation of concerns and smooth interaction between components.

2.2 Components Design

The system is composed of the following major components:

- File Scanning Component
- Scans all files in the retail directory and extracts metadata such as file name, size, and timestamps.
- Hash Generation Component
- Generates SHA-256 hash values for each file to ensure accurate integrity verification.
- Baseline Management Component
- Stores the initial state of files as a baseline for future comparison.
- Comparison Engine
- Compares current file data with baseline data to detect modifications, additions, or deletions.
- Alert Generation Component
- Identifies suspicious changes and generates alerts with assigned risk levels.
- Report Generation Component
- Creates CSV-based forensic reports containing detected changes and file details.
- Dashboard Component
- Displays system status, scan results, alerts, and visualizations through the user interface.

2.3 Process Flow

The system follows a structured sequence of operations:

1. User initiates file scanning through the dashboard
2. System scans files and extracts metadata
3. Hash values are generated for each file
4. Baseline is created or loaded
5. User performs data modifications (simulation)
6. System re-scans files
7. Comparison engine detects changes

8.Alerts are generated for suspicious files
9.Results are displayed in the dashboard
10.CSV report is generated for forensic analysis

This flow ensures continuous monitoring and accurate detection of file integrity violations.

2.4 Information Flow

The flow of information in the system is as follows:
- User interacts with the Streamlit dashboard
- Dashboard sends request to backend logic
- Backend accesses files from the monitored directory
- Metadata and hash values are processed
- Results are stored in CSV/JSON files
- Processed data is sent back to the dashboard
- Dashboard displays analysis, alerts, and reports

This ensures smooth communication between user interface, processing engine, and data storage.

3. Module Design (High-Level)

3.1 File Integrity Scanning Module

This module scans all files within the selected retail directory and collects essential metadata such as file name, path, size, and timestamps. It also computes a unique hash value for each file to support integrity verification.
The output of this module serves as the current state of the system, which is used for further analysis and comparison.

3.2 Baseline Management Module

This module is responsible for creating and maintaining a baseline of file states. The baseline represents a trusted snapshot of files captured during an initial scan.
It allows the system to:
- Store baseline data for future reference
- Load existing baseline data
- Ensure consistency during comparison

The baseline acts as the foundation for detecting any unauthorized changes.

3.3 Comparison & Detection Module

This module compares the current scan results with the stored baseline data. It identifies differences such as:
- Modified files
- Newly added files
- Deleted files

Based on the comparison, each file is assigned a status indicating whether it has been altered or remains unchanged.

3.4 Alert & Risk Detection Module

This module analyzes detected changes and determines their significance. It generates alerts for suspicious file modifications and assigns risk levels such as low or medium.
These alerts help users quickly identify potential security issues and focus on critical changes.

3.5 Reporting Module

This module generates forensic reports in CSV format. The reports include details such as file metadata, detected changes, timestamps, and risk levels.
Users can download these reports for further investigation, documentation, or audit purposes.

3.6 Dashboard & Visualization Module

This module provides the user interface through a Streamlit-based dashboard. It displays:

- File scan results in tabular format
- Summary statistics (total files, modified files, etc.)
- Visual charts and timelines
- Alerts and system status

It ensures that users can easily understand system outputs and monitor file integrity.

4. Data Design

4.1 Data Model

The system uses a structured data model to store file information and analysis results. Each file entry contains the following fields:

- file_name – Name of the file
- file_path – Location of the file
- size – File size
- created_time – File creation timestamp
- modified_time – Last modified timestamp
- hash_value – Unique hash used for integrity verification
- status – File state (Normal / Modified / New / Deleted)
- risk_level – Risk classification (Low / Medium)
-

This model ensures consistent representation of file data for comparison and reporting.

4.2 Data Storage

The system uses simple file-based storage for maintaining data:

- CSV Files
- Used for storing generated reports containing file analysis results.
- JSON Files
- Used for storing baseline data, including file metadata and hash values.
- Text Logs

- Used to record system activity and execution details.

This approach keeps the system lightweight and easy to manage.

4.3 Data Retention Policies

- Baseline data is preserved for future comparisons unless manually updated.
- Generated reports are stored locally and can be accessed anytime.
- Logs are maintained for tracking system activity.

This ensures availability of historical data for forensic analysis.

4.4 Data Migration

The current system uses local file storage (CSV/JSON). For future scalability, the data can be migrated to a database system such as:

- MySQL
- PostgreSQL
- MongoDB

This would allow better data management, querying, and scalability.

5. Interfaces

5.1 User Interface (Dashboard)

- The system provides a web-based user interface built using Streamlit. The dashboard allows users to interact with the system and perform various operations related to file integrity monitoring.
- The interface includes:
- File Scanning Panel
- Allows users to initiate scanning of retail files and view file metadata.
- File Integrity Analysis Table
- Displays scan results including file status, timestamps, and risk levels.
- Baseline Management Controls
- Enables users to create and load baseline data for comparison.
- Retail Data Panel
- Allows users to view and manage customer or product data.
- Simulation Panel
- Provides options to simulate real-world changes such as adding or modifying data.
- Alerts Section
- Displays warnings for detected file tampering.
- Report Download Option

- Allows users to export results as CSV files for forensic analysis.
- The interface is designed to be simple, interactive, and easy to use.
- 5.2 File System Interface
- The system interacts directly with the local file system to monitor and analyze files.
- This includes:
- Accessing files from the monitored retail directory
- Reading file metadata such as size and timestamps
- Generating hash values for integrity verification
- Updating baseline data stored in JSON format
- Writing reports and logs to local storage
- The system ensures that all file operations are performed in a read-only manner to maintain data integrity.

6.Non-Functional Requirements

6.1 Security

- The system is designed to preserve the integrity of evidence and prevent unintended data modification.
- Read-only file access: Files are scanned without altering original content.
- Hash-based integrity verification: File hashes are used to accurately detect tampering.
- Secure local storage: Baseline data and reports are stored locally and not exposed externally.
- Controlled access (demo scope): Intended to run on a local environment during demonstration.
- These measures ensure that the system can be used for reliable forensic analysis.

6.2 Performance

- The system is optimized for small to medium-sized datasets typical of academic or demo environments.
- Efficient scanning: Processes files sequentially with minimal overhead.
- Lightweight storage: Uses CSV/JSON, avoiding heavy database dependencies.
- Responsive UI: Streamlit dashboard updates results quickly after each operation.
- Performance may vary based on the number and size of files being scanned.

6.3 Limitations

- No real-time monitoring; scans are triggered manually.
- Limited scalability due to file-based storage.
- Designed for local/demo use; no cloud integration.
- Basic risk classification (can be enhanced further).

6.4 Future Enhancements

- Add real-time monitoring and automated alerts
- Integrate with cloud storage (AWS, Azure)
- Use database systems for scalable storage
- Enhance analytics and visualization (advanced charts)
- Add user authentication and access control

7. Limitations & Future Enhancements

7.1 Limitations

The current system has the following limitations:

- The system does not support real-time monitoring; scans must be initiated manually.
- It operates only in a local environment and does not support cloud-based file monitoring.
- Data storage is limited to CSV and JSON files, which may not be suitable for large-scale systems.
- The risk detection mechanism is basic and does not include advanced threat analysis.
- No user authentication or access control is implemented.
- The system is designed primarily for academic and demonstration purposes.
-

7.2 Future Enhancements

The system can be further improved with the following enhancements:

- Implement real-time file monitoring using file watchers to detect changes instantly.
- Integrate automated alert systems such as email, SMS, or dashboard notifications.
- Extend support for cloud environments (AWS, Azure, Google Cloud) to monitor distributed storage systems.
- Replace file-based storage with scalable databases like MySQL, PostgreSQL, or MongoDB.
- Incorporate machine learning techniques for intelligent anomaly detection and risk prediction.
- Add user authentication and role-based access control for secure multi-user access.
- Enhance the dashboard with advanced visualizations, timelines, and detailed analytics.
- Enable integration with SIEM tools for centralized security monitoring.
- Improve the system to support real-time logging and audit trails for better forensic tracking.
- Provide support for API-based integration with other enterprise systems.

8. References

The following resources were used during the design and development of the system:

- Python Official Documentation – https://docs.python.org/3/
- Streamlit Documentation – https://docs.streamlit.io/
- hashlib (Python Hashing Library) – https://docs.python.org/3/library/hashlib.html
- CSV Module Documentation – https://docs.python.org/3/library/csv.html
- JSON Module Documentation – https://docs.python.org/3/library/json.html
- Digital Forensics Concepts – Online learning resources and academic materials
- File Integrity Monitoring (FIM) Concepts – Cybersecurity study references